

Computación de Altas Prestaciones
Máster Universitario en Ingeniería Informática
Enseñanzas Prácticas: Despliegue de Apache Spark

Objetivos

1. Introducción al manejo de datos masivos.
2. Introducción al uso de clusters Apache Spark

Antes de comenzar

Este documento se ha realizado para el despliegue de clusters Spark en Linux. Las versiones Mint y Ubuntu instaladas en los laboratorios están verificadas, por lo que se recomienda trabajar con alguna de estas distribuciones.

Esta práctica se basará en la instalación y configuración de la plataforma Spark y ejecutar una aplicación en modo local. Además, se detallará los pasos para configurar un clúster Standalone de Spark formado por un maestro y un esclavo, lanzando el algoritmo KMeans de Spark como ejemplo de funcionamiento. En la segunda parte de la práctica, se trabajará sobre la configuración del sistema de archivos de Hadoop (HDFS) para lanzar aplicaciones.

Desarrollo de la práctica

1. Descarga e instalación de paquetes necesarios

Se presentan aquí todos los pasos necesarios para la realización de la práctica, suponiendo que se parte de una imagen Linux limpia. Nótese que algunos pasos pueden que necesiten ser obviados en caso de que vaya a realizarlo en sus equipos personales.

1.1 Instalación de JDK

```
$ sudo apt update
$ sudo apt upgrade
$ sudo apt install default-jdk
```

1.2 Instalación de Scala

```
$ sudo apt-get install scala
```

1.3 Instalación de Spark

Para la realización de esta práctica, se necesitará descargar el siguiente paquete: Spark-3.2.1.-bin-hadoop3.2 (https://archive.apache.org/dist/spark/spark-3.2.1/spark-3.2.1-bin-hadoop3.2.tgz)

```
$ tar -xvf spark-3.2.1-bin-hadoop3.2.tar.gz
$ sudo mv spark-3.2.1-bin-hadoop3.2 /opt/spark
$ sudo chown -R usuario /opt/spark
```

1.4 Instalación de OpenSSH

```
$ sudo apt install openssh-server
```

1.5 Configurar variable de entorno

```
$ cd
$ nano .bashrc
$ export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
$ source ~/.bashrc
```

Al ser el worker una copia del master (recordemos que más adelante copiamos la máquina virtual sobre la que trabajamos) no es necesario definer las variables de entorno de Spark, aún así, en caso de obtener error o por asegurarnos, añadimos al archivo anterior bashrc, las siguientes líneas (recordar ejecutar después source ~/.bashrc para activar las variables añadidas):

```
$ export SPARK_HOME=/opt/spark
$ export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

1.6 Clonación de las máquinas virtuales

Este paso debe realizarse si se quiere desplegar Apache Spark en dos máquinas virtuales. Si tal es el caso, debe apagar la máquina virtual, ya que no se pueden clonar máquinas si la de origen está encendida. Una vez esté apagada, se hará una clonación de tipo enlazada (linked). Llame a la máquina original **master** y a la nueva máquina virtual clonada: **worker.**

2. Montando el cluster

A continuación, vamos a configurar un clúster de 2 máquinas (a partir de ahora, maestro y esclavo). Para ello, necesitamos dos máquinas (que pueden ser virtuales), ambas con Java, Spark y Scala ya instalados y que se encuentren en la misma red.

2.1 Intercambio de claves

Apache Spark necesita acceder por SSH a las máquinas, pero sin contraseña, por lo que habrá que crear una clave para permitirle establecer la conexión.

En el nodo **master**, generamos la clave RSA y la vinculamos al nodo **worker** (cuando ejecutemos el comando, dejamos los valores por defecto a todo lo que nos solicite):

```
$ ssh-keygen
$ ssh-copy-id -i ~/.ssh/id_rsa.pub usuarioWorker@IPWorker
$ ssh-copy-id -i ~/.ssh/id_rsa.pub usuarioMaster@IPMaster
```

2.2 Configuración del cluster

Una vez realizado esto, debemos configurar Spark para identificarle quién es el nodo maestro, quién el esclavo y las características de cada worker. Para ello, tanto en el nodo **maestro** como en el **worker**, tendremos que configurar dos archivos (slaves y spark-env.sh):

```
$ cd /opt/spark/conf/
$ nano slaves
$ cp spark-env.sh.template spark-env.sh
$ nano spark-env.sh
```

El contenido del fichero *slaves* debe ser:

```
# A Spark Worker will be started on each of the machines listed below.
# localhost
IP DEL NODO WORKER
```

Las líneas que se deben añadir en el fichero *spark-env.sh* son:

```
export SPARK_MASTER_HOST=IP_NODO_MAESTRO
export SPARK_WORKER_CORES=1
export SPARK_WORKER_MEMORY=1g
```

Con el fichero anterior, establecemos las características del cluster. En este caso, hemos establecido 1 core por worker, de 1 GB de memoria cada uno. Como se puede ver en el fichero, podemos configurar diversos parámetros, como el puerto, el directorio donde se ubicarán los proyectos u otros parámetros.

2.3 Desplegando el cluster desde el nodo master

Para lanzar el clúster, tendremos que iniciar todos los componentes del mismo, para ello, tenemos dentro del directorio `sbin` de Spark los diversos ejecutables para iniciar o parar los nodos del clúster.

Acto seguido, levantamos todos los nodos del cluster (realizar desde el nodo master):

```
$ cd /opt/spark/sbin/
$ ./start-all.sh
```

Si ahora nos vamos al monitor del sistema en el nodo worker, podremos ver que hay en ejecución dos procesos de Java correspondientes al cluster Spark (ver Figura 1).

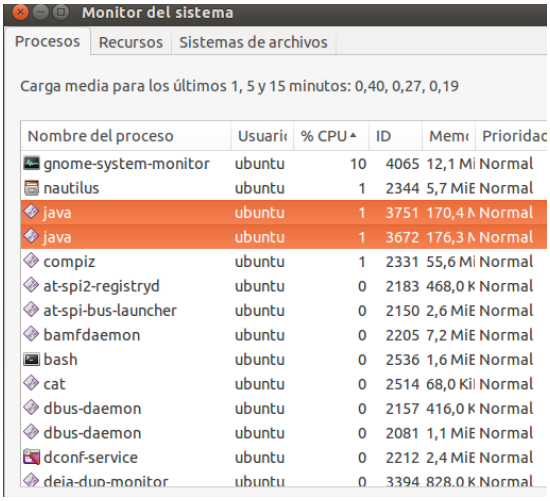


Figura 1. Procesos del cluster en ejecución.

3. Explotación del cluster

3.1 Interfaz gráfica del clúster

Apache Spark ofrece información acerca del rendimiento del cluster. Igualmente, muestra alguna información de su funcionamiento en tiempo real. Podemos tener dicha información del cluster si accedemos a la dirección http://IP_MASTER:8080/ desde el navegador, obteniendo una visión similar a la mostrada en la Figura 2.

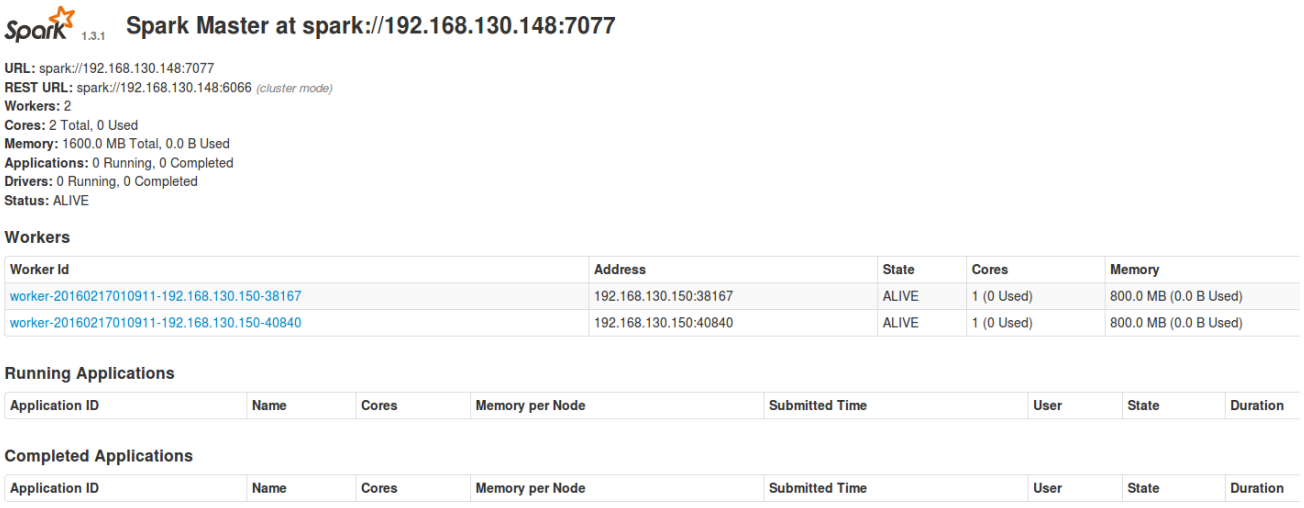


Figura 2. Interfaz gráfica de Apache Spark, desde el master.

Se puede ver cómo está montando el cluster en la dirección 192.168.130.148:7077 (IP-MASTER:7077), que será donde tendremos que lanzar nuestras aplicaciones. Podemos diferenciar también las instancias en la tabla de workers, donde las podemos diferenciar por el identificador o por el puerto.

3.2 Lanzar una aplicación de test

Para comprobar el funcionamiento del cluster, vamos a lanzar el algoritmo KMeans de Spark en Scala. Para ello, desde el nodo master, y estando en el directorio de Spark, lanzaremos la aplicación al cluster:

```
$ cd /opt/spark

$ ./bin/spark-submit --master spark://192.168.130.148:7077 --class
org.apache.spark.examples.mllib.KMeansExample examples/jars/spark-examples_2.12-3.2.1.jar
```

Si visualizamos el navegador, observaremos como se está ejecutando la aplicación y los detalles de la misma como el nombre, el número de cores, la memoria por nodo, el usuario o el tiempo que lleva ejecutándose, tal y como se muestra en la Figura 3.

Running Applications

Application ID	Name	Cores	Memory per Node	Submitted Time	User	State	Duration
app-20160217012236-0000	Spark shell	2	512.0 MB	2016/02/17 01:22:36	ubuntu	RUNNING	1,5 min

Figura 3. Aplicación corriendo sobre Apache Spark.

Si hacemos click en uno de los nodos, podremos ver lo que está ejecutando (ver Figura 4).

Spark Worker at 192.168.130.150:38167

ID: worker-20160217010911-192.168.130.150-38167
Master URL: spark://192.168.130.148:7077
Cores: 1 (1 Used)
Memory: 800.0 MB (512.0 MB Used)

Back to Master

Running Executors (1)

ExecutorID	Cores	State	Memory	Job Details	Logs
0	1	LOADING	512.0 MB	**ID:** app-20160217012236-0000 **Name:** Spark shell **User:** ubuntu	stdout stderr

Figura 4. Detalles de ejecución de cada worker.

Al salir de la aplicación, podremos observar cómo la tarea que estaba ejecutándose pasa a la lista de finalizadas (Figura 5).

Completed Applications

Application ID	Name	Cores	Memory per Node	Submitted Time	User	State	Duration
app-20160217012236-0000	Spark shell	2	512.0 MB	2016/02/17 01:22:36	ubuntu	FINISHED	5,0 min

Figura 5. Ejemplo de tareas ya finalizadas.